Système d'exploitation : TD2

février 2026 — Pierre Rousselin

Exercice 1 : Qui dit quoi ?

On considère le programme suivant `a.c` (où on a omis les directives `#include` par soucis de concision).

```c
int main(int argc, char *argv[])
{
        if (argc < 2) {
                fprintf(stderr, "Usage: a arg...\n");
                return 1;
        }
        printf("Argument 1 : %s\n", argv[1]);

        char **new_args = malloc(argc * sizeof(char *));
        memcpy(new_args + 1, argv + 2, (argc - 1) * sizeof(char *));
        new_args[0] = "/bin/hello";

        switch (fork()) {
        case -1:
                perror("fork");
                return 1;
        case 0:
                execv("/bin/echo", new_args);
                printf("bar\n");
                exit(1);
        }
        printf("foo\n");
        wait(NULL);
        free(new_args);
        printf("baz\n");
        return 0;
}
```

On suppose qu'on a compilé et exécuté ce programme avec les commandes suivantes

```
$ gcc -Wall -Wvla -std=c99 -g a.c -o a
$ ./a plop hop hello
```

1. Faire un dessin des tableaux `argv`, `new_args` juste avant le `fork`. Pour rappel, le prototype et la spécification de `memcpy` sont les suivant :

   ```c
   void *memcpy(void *restrict s1, const void *restrict s2, size_t n);
   ```

   ```
   The  memcpy() function shall copy n bytes from the object pointed to by
   s2 into the object pointed to by s1. If copying takes place between objects
   that overlap, the behavior is undefined.
   ```

   (Vous pouvez ignorer le mot-clé `restrict`.)
2. Le programme `/bin/echo` affiche tous ses arguments, séparés par un espace, puis une fin de ligne. Dire ce qui est affiché par ce programme et dans quel ordre. Si il y a situation de compétition pour des affichages, dire dans quel cas.
3. Que se passe-t-il si dans l'appel à `execv`, `"/bin/echo"` est remplacé par `"a"` ?
4. Même question, mais en remplaçant `execv` par `execvp`, qui utilise le `PATH` pour la recherche de fichiers exécutables si son premier argument ne contient pas de caractère oblique (`/`).

--- * ---

Exercice 2 : Une *forkbomb* élaborée

On considère le programme suivant (où les directives `#include` ont été omises), qu'on a compilé de la façon suivante :

```
$ gcc -Wall encore_plus_de_fork.c -o encore
```

```c
#define N_DFT 2
#define P_DFT 3
#define FMT_PROC "pid : %d; ppid : %d; n : %d; p : %d\n"

int main(int argc, char *argv[])
{
        int n = argc < 2 ? N_DFT : atoi(argv[1]);
        int p = argc < 3 ? P_DFT : atoi(argv[2]);
        int i;

        if (p == 0) {
                printf("Sortie 1\n");
                return 0;
        }

        for (i = 0; i < n; ++i)
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        printf(FMT_PROC, getpid(), getppid(), n, p);
                        char str_n[11], str_p[11];
                        sprintf(str_n, "%d", n);
                        sprintf(str_p, "%d", p - 1);
                        execl("encore", "encore", str_n, str_p, NULL);
                        printf("Sortie 2\n");
                        return 0;
                }

        while (wait(NULL) != -1)
                ;
        printf("Sortie 3\n");
        return 0;
}
```

1. Dans le cas où le programme est lancé de la façon suivante :

   ```
   $ ./encore
   ```

   dessiner l'arborescence des processus et donner le nombre de `Sortie 1`, `Sortie 2` et `Sortie 3` qui seront affichés.

2. Soit $u_{n,p}$ le nombre de processus créés par ce programme lorsque les arguments entiers naturels n et p lui sont transmis. Donner la relation entre $u_{n,p}$ et $u_{n,p-1}$ lorsque $p > 0$. Puis calculer $u_{n,p}$. En déduire le nombre d'affichages de `Sortie 1`, `Sortie 2` et `Sortie 3` en fonction de n et p.

--- * ---